UT3 - Tarea 2: Práctica Avanzada de Gestión de Identidades, Auditoría y Automatización en MySQL 8.0

1. CONSIDERACIONES PREVIAS

Escenario Profesional (Contexto Real de Producción)

Trabajas como Administrador de Sistemas y Bases de Datos (SysAdmin/DBA) en **SanaSalud S.A.**, una empresa proveedora de servicios médicos de alta disponibilidad. El SGBD corporativo (MySQL 8.0) custodia información clínica y datos personales extremadamente sensibles sometidos al Reglamento General de Protección de Datos (RGPD) y a la Ley Orgánica de Protección de Datos y Garantía de Derechos Digitales (LOPDGDD).

Recientemente se ha detectado una fuga de credenciales debido a la existencia de scripts con contraseñas en texto claro (*hardcoded*) y cuentas con permisos globales desproporcionados. La dirección de sistemas te encarga la reestructuración completa del sistema de control de accesos e identidades bajo las siguientes premisas:

1. **Políticas de endurecimiento (Hardening) de credenciales.**
2. **Seguridad basada en Roles (RBAC) para el personal de la clínica.**
3. **Integración segura de un Script de Monitorización automatizado en Python sin credenciales expuestas.**
4. **Auditoría periódica de las tablas de diccionario del sistema.**

2. ACTIVIDADES A REALIZAR

Fase 1: Políticas de Contraseñas y Bloqueos (Hardening)

Como primera medida de mitigación de riesgos, debes activar e implementar políticas estrictas sobre el ciclo de vida de las credenciales de los usuarios utilizando los componentes nativos de MySQL 8.0.

Requisitos Técnicos:

Escribe un script SQL que configure una cuenta de usuario administrativa llamada sec_auditor@localhost que cumpla obligatoriamente con las siguientes reglas de hardening:

- Debe expirar de forma forzosa cada **90 días** para obligar a su rotación.
- Si se introduce de forma errónea la contraseña **3 veces consecutivas**, la cuenta debe bloquearse de forma automática y temporal durante **5 días** para evitar ataques de fuerza bruta.

- No se debe permitir la reutilización de ninguna de las últimas **3 contraseñas** históricas del usuario, ni claves que tengan menos de **365 días** de antigüedad.

Script SQL de Hardening (Fase 1):

```
-- Creación y configuración de la cuenta con políticas de Hardening integradas
CREATE USER IF NOT EXISTS 'sec_auditor'@'localhost'
IDENTIFIED BY 'SanaSecure#2026'
PASSWORD EXPIRE INTERVAL 90 DAY
FAILED_LOGIN_ATTEMPTS 3 PASSWORD_LOCK_TIME 5
PASSWORD REUSE INTERVAL 365 DAY
PASSWORD REUSE HISTORY 3;

-- Modificación de parámetros globales del servidor para hacer estas políticas obligatorias
-- (Opcional, requiere privilegios de SYSTEM_VARIABLES_ADMIN o SUPER)
SET GLOBAL default_password_lifetime = 90;
SET GLOBAL disconnect_on_expired_password = ON;
```

Fase 2: Implementación de Seguridad Basada en Roles (RBAC)

Para evitar la asignación caótica de permisos individuales, estructurarás el acceso del personal de la clínica mediante el control de accesos basado en roles (RBAC) sobre la base de datos corporativa sanasalud_db (la cual consta de las tablas pacientes, historiales_clinicos, personal_medico y citas_medicas).

Definición de Roles y Permisos:

1. **rol_sanitario:** Acceso de lectura (SELECT) a la información general de pacientes y citas para coordinar consultas, y permisos completos de manipulación (SELECT, INSERT, UPDATE) sobre historiales_clinicos para la práctica asistencial. No debe tener permisos de borrado (DELETE).
2. **rol_administrativo:** Permisos completos (SELECT, INSERT, UPDATE, DELETE) en las tablas pacientes y citas_medicas para la gestión administrativa de admisiones. No debe tener visibilidad ni acceso de lectura a la tabla historiales_clinicos.
3. **rol_auditor_seguridad:** Permiso de lectura exclusiva (SELECT) en todas las tablas del esquema sanasalud_db y acceso a las vistas de monitorización del rendimiento.

Tarea del Alumno:

- Crear los roles indicados.
- Crear tres cuentas de usuario reales: doctor_gomez@localhost, recepcionista_perez@localhost y agente_auditor@localhost.
- Vincular cada cuenta a su respectivo rol de seguridad.

- Garantizar la activación automática de dichos roles al iniciar la sesión del usuario (SET DEFAULT ROLE).

Solucionario de la Configuración RBAC (Fase 2):

```
-- 1. Creación de la base de datos de pruebas para el escenario
CREATE DATABASE IF NOT EXISTS sanasalud_db;
USE sanasalud_db;

-- Creación física de las tablas del escenario
CREATE TABLE IF NOT EXISTS pacientes (
id_paciente INT AUTO_INCREMENT PRIMARY KEY,
nombre VARCHAR(100),
nif VARCHAR(9) UNIQUE,
telefono VARCHAR(15)
);

CREATE TABLE IF NOT EXISTS citas_medicas (
id_cita INT AUTO_INCREMENT PRIMARY KEY,
id_paciente INT,
fecha_hora DATETIME,
box VARCHAR(10),
FOREIGN KEY (id_paciente) REFERENCES pacientes(id_paciente)
);

CREATE TABLE IF NOT EXISTS historiales_clinicos (
id_historial INT AUTO_INCREMENT PRIMARY KEY,
id_paciente INT,
diagnostico TEXT,
tratamiento TEXT,
fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (id_paciente) REFERENCES pacientes(id_paciente)
);

-- 2. Creación de los Roles
CREATE ROLE IF NOT EXISTS 'rol_sanitario', 'rol_administrativo', 'rol_auditor_seguridad';

-- 3. Dotación de privilegios a los Roles
-- Rol Sanitario
GRANT SELECT ON sanasalud_db.pacientes TO 'rol_sanitario';
GRANT SELECT ON sanasalud_db.citas_medicas TO 'rol_sanitario';
GRANT SELECT, INSERT, UPDATE ON sanasalud_db.historiales_clinicos TO 'rol_sanitario';

-- Rol Administrativo
GRANT SELECT, INSERT, UPDATE, DELETE ON sanasalud_db.pacientes TO
'rol_administrativo';
GRANT SELECT, INSERT, UPDATE, DELETE ON sanasalud_db.citas_medicas TO
'rol_administrativo';

-- Rol Auditor de Seguridad
GRANT SELECT ON sanasalud_db.* TO 'rol_auditor_seguridad';

-- 4. Creación de usuarios reales
```

```
CREATE USER 'doctor_gomez'@'localhost' IDENTIFIED BY 'DocGomezPass#2026';
CREATE USER 'recepcionista_perez'@'localhost' IDENTIFIED BY 'RecepPerezPass#2026';
CREATE USER 'agente_auditor'@'localhost' IDENTIFIED BY 'AgentAuditPass#2026';

-- 5. Asignación de Roles a los usuarios
GRANT 'rol_sanitario' TO 'doctor_gomez'@'localhost';
GRANT 'rol_administrativo' TO 'recepcionista_perez'@'localhost';
GRANT 'rol_auditor_seguridad' TO 'agente_auditor'@'localhost';

-- 6. Configuración de Activación Automática por Defecto (Muy importante)
SET DEFAULT ROLE 'rol_sanitario' TO 'doctor_gomez'@'localhost';
SET DEFAULT ROLE 'rol_administrativo' TO 'recepcionista_perez'@'localhost';
SET DEFAULT ROLE 'rol_auditor_seguridad' TO 'agente_auditor'@'localhost';

-- Sincronizar diccionario
FLUSH PRIVILEGES;
```

Fase 3: Integración y Automatización Segura con Python (PAR/IAW Interdisciplinar)

Uno de los errores más graves en entornos de producción es programar scripts en Bash, Python o PHP que contienen la contraseña del usuario administrador (root) en texto claro (*hardcoded*).

Para resolver esta vulnerabilidad, diseñarás una solución profesional:

1. Crearás una cuenta de servicio de mínimos privilegios en MySQL llamada monitor_agent@localhost.
2. Implementarás un script en Python que recupera la credencial de conexión de forma dinámica a través de una **variable de entorno del sistema operativo**, evitando escribir claves en los archivos de código fuente.

Configuración del Usuario de Servicio (SQL):

```
-- Usuario exclusivo para el microservicio de monitorización
CREATE USER 'monitor_agent'@'localhost' IDENTIFIED BY 'SanaMonitorAgentPass2026#';

-- Conceder accesos de sólo lectura a las tablas de metadatos del motor
GRANT SELECT ON information_schema.* TO 'monitor_agent'@'localhost';
GRANT SELECT ON sanasalud_db.* TO 'monitor_agent'@'localhost';
```

Script de Python para Monitorización de Almacenamiento (monitor_space.py):

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
SanaSalud S.A. - Script de Monitorización de Almacenamiento Seguro
Módulo: ASGBD / IAW (ASIR)
Este script demuestra las mejores prácticas de SysAdmin:
```

```
- Recupera las credenciales de conexión desde variables de entorno del S.O.
- Conecta usando un usuario de mínimos privilegios ('monitor_agent').
- Previene fugas de contraseñas por exposición de archivos de código fuente.
"""

import os
import sys
import mysql.connector
from mysql.connector import Error

def obtener_credenciales():
"""Recupera los parámetros de conexión y credenciales desde el entorno del S.O."""
db_host = os.getenv('DB_SANA_HOST', 'localhost')
db_user = os.getenv('DB_SANA_USER', 'monitor_agent')
db_pass = os.getenv('DB_SANA_PASS') # Contraseña almacenada de forma segura en el
entorno
db_name = os.getenv('DB_SANA_NAME', 'sanasalud_db')

if not db_pass:
print("[!] ERROR CRÍTICO: La variable de entorno 'DB_SANA_PASS' no está definida.",
file=sys.stderr)
print("[i] Para definirla en Linux/macOS, ejecuta de antemano en tu terminal:\n",
file=sys.stderr)
print(" export DB_SANA_PASS='SanaMonitorAgentPass2026#'\n", file=sys.stderr)
print("[i] Para definirla en Windows (Command Prompt), ejecuta:\n", file=sys.stderr)
print(" set DB_SANA_PASS=SanaMonitorAgentPass2026#\n", file=sys.stderr)
sys.exit(1)

return db_host, db_user, db_pass, db_name

def monitorizar_almacenamiento():
host, user, password, database = obtener_credenciales()
connection = None
try:
connection = mysql.connector.connect(
host=host,
user=user,
password=password,
database=database
)

if connection.is_connected():
cursor = connection.cursor(dictionary=True)

# Consulta avanzada al diccionario de datos para medir el peso de cada tabla de la clínica
query = """
SELECT
table_name AS 'Tabla',
ROUND(((data_length + index_length) / 1024 / 1024), 4) AS 'Tamano_MB',
table_rows AS 'Filas'
FROM information_schema.TABLES
WHERE table_schema = %s
ORDER BY (data_length + index_length) DESC;
"""
```

```
cursor.execute(query, (database,))
tablas = cursor.fetchall()

print("=================================================================")
print(f" REPORTE DE USO DE ESPACIO EN DISCO: {database.upper()}")
print("=================================================================")
for tabla in tablas:
print(f" -> Tabla: {tabla['Tabla']:<25} | Filas: {tabla['Filas']:<8} | Tamaño: {tabla['Tamano_MB']:>8}
MB")
print("=================================================================")

cursor.close()

except Error as e:
print(f"[!] Error de conexión a la base de datos: {e}", file=sys.stderr)
finally:
if connection and connection.is_connected():
connection.close()
print("[i] Conexión cerrada de forma segura por el agente de monitorización.")

if __name__ == "__main__":
monitorizar_almacenamiento()
```

Fase 4: Auditoría Avanzada de Cuentas mediante Catálogos del Sistema

Como Auditor de Seguridad / DBA, tu tarea no es sólo configurar los accesos, sino también programar consultas de supervisión de forma periódica contra el catálogo interno de MySQL para detectar posibles agujeros de seguridad.

Consultas de Auditoría requeridas:

1. Detección de comodines inseguros de red

Muestra todos los usuarios y roles configurados que permiten el acceso desde cualquier ubicación remota (%), omitiendo las cuentas legítimas autorizadas:

```
SELECT user, host, plugin
FROM mysql.user
WHERE host = '%' AND user NOT IN ('root', 'mysql.sys');
```

2. Localización de algoritmos obsoletos o débiles

Localiza cuentas que sigan utilizando el algoritmo heredado de hash mysql_native_password en lugar de la encriptación moderna SHA-256 (caching_sha2_password):

```
SELECT user, host, plugin, password_last_changed
FROM mysql.user
WHERE plugin = 'mysql_native_password';
```

3. Auditoría de asignación y herencia de Roles (Mapeo de privilegios)

Muestra un reporte claro que relacione los usuarios de la base de datos con los roles que tienen asociados y el estado de activación por defecto:

```
SELECT
FROM_USER AS 'Rol_Origen',
TO_USER AS 'Usuario_Destino',
WITH_ADMIN_OPTION AS 'Puede_Delegar'
FROM mysql.role_edges;
```

4. Identificación de cuentas bloqueadas o con claves expiradas

Lista los usuarios que tienen suspendido el acceso temporal o permanentemente por motivos de seguridad:

```
SELECT user, host, account_locked, password_expired, password_lifetime
FROM mysql.user
WHERE account_locked = 'Y' OR password_expired = 'Y';
```

4. CALIFICACIÓN

> ⚠️ **IMPORTANTE:** El retraso en el plazo de entrega a través de la plataforma, la entrega del documento en formatos editables no permitidos (como .docx), capturas con identificación del alumno/a o la omisión de los enlaces de las fuentes documentales utilizadas penalizará gravemente la nota o invalidará la práctica.

Sección Evaluada	Descripción Técnica Exigida al Alumno	Puntuación
Fase 1: Hardening	Implementación exacta de la directiva de intentos fallidos, intervalo de reuso de contraseñas e histórico de contraseñas en MySQL.	**2.5 pts**
Fase 2: RBAC	Creación limpia de roles y asignación selectiva a los tres perfiles de la clínica sin solapamientos. Activación automática mediante SET DEFAULT ROLE.	**2.5 pts**
Fase 3: Automatización	Puesta en marcha de variables de entorno en el sistema operativo del alumno y ejecución sin errores del script en Python.	**3.0 pts**

Sección Evaluada	Descripción Técnica Exigida al Alumno	Puntuación
Fase 4: Consultas Auditoras	Ejecución correcta de consultas SQL contra la base de datos mysql para reportar riesgos y metadatos de accesos.	**2.0 pts**